AMax Firmware Update — Quick Guide (gant)

How to make delila-rs (the DAQ + Operator UI) follow a new AMax firmware register map. **Everything runs locally on this machine — no `git commit` / `git push` is required.** You edit/drop in a `RegisterFile.json`, run one script, restart the DAQ, and you're done.

> Maintainers who want to share the result with other machines handle that separately. As an AMax FW developer you only need the steps below.

---

1. What the one command does

```
RegisterFile.json
   │  amax_codegen   (register addresses auto-derived — no flags to guess)
   ├─→ src/config/amax_generated.rs                    (AMaxChannelConfig
struct)
   ├─→ src/reader/caen/amax_registers_generated.rs     (REG_*,
BROADCAST_BASE, merge fn)
   └─→ web/operator-ui/src/app/models/amax-generated.ts (UI types + param
defs + tabs)
         │  cargo fmt → cargo build → ng build
         └─→ web/operator-ui/dist/                      (the UI bundle
that is served)
```

`PAGE_BASE` / `PAGE_STRIDE` / `BROADCAST_BASE` are all derived from `RegisterFile.json` automatically. You never hand-pick them, and you never edit `handle.rs`.

---

2. Prerequisites (already set up on gant)

Tool	Notes
`cargo`	`source ~/.cargo/env`
`node` + `npm`	Node 22 LTS in `~/.local/node`, on `PATH`. `web/operator-ui/node_modules` already installed.
Input	`RegisterFile.json` from the FW build
UI metadata	`tools/amax_viewer/fw_params.json` (label / category / type / bit width / default per register)

---

3. Quick start (normal case — addresses moved)

```
cd /media/raid1/delila-rs
source ~/.cargo/env
```

```
# 1. Put the new RegisterFile.json somewhere in the repo (convention:
FW/<date>/)
#    e.g. FW/20260701/RegisterFile_1july.json

# 2. One command
scripts/update_amax_fw.sh FW/20260701/RegisterFile_1july.json
```

You'll see a summary:

```
amax_codegen layout: PAGE_BASE=0x... (auto), PAGE_STRIDE=0x... (auto),
BROADCAST_BASE=0x... (auto), canonical=per-channel ch0
amax_codegen: register set unchanged (29 registers)
```

If it says `register set unchanged`, you're done with regeneration — go to **§6 Apply it (restart the DAQ)**.

> The script prints a `git add ...` hint at the very end. **You can ignore it** on gant — your changes are already built and stay local.

Options

- `--no-ui` — skip the Angular build (Rust-only, faster)
- `--with-viewer` — also regenerate amax_viewer's `register_defs.json` (best-effort)
- `--help` — usage

---

4. When the register set changes

Added / removed registers

The summary tells you:

```
amax_codegen: register set changed — 2 added, 0 removed
  + NEW_REG_A, NEW_REG_B
warning: 2 register(s) in RegisterFile.json have no entry in fw_params.json
(skipped):
  - NEW_REG_A
  - NEW_REG_B
```

`no entry in fw_params.json (skipped)` means **that register is NOT exposed** (neither typed config nor UI). To expose it, add one line to `tools/amax_viewer/fw_params.json` under `params`:

```
"NEW_REG_A": { "bits": 16, "default": 100, "label": "New Reg A",
"category": "energy", "type": "number", "unit": "ns" }
```

Field	Meaning
`bits`	data bit width (`max = 2^bits − 1` computed automatically)
`default`	initial value
`label`	UI display name
`category`	UI tab: `input` / `trigger` / `energy` / `amax`, **or a new name** (see below)
`type`	`number`, or `enum` with `options: [...]`
`unit`	optional display unit

Then run the same command again. Removing a register needs no metadata edit — codegen shrinks the struct and the merge function together (no `handle.rs` edit).

New category (new tab)

Just use a `category` name that doesn't exist yet (e.g. `debug`). The codegen emits the new param array and category map, and the Operator UI shows a **new tab automatically** (Settings and Tune Up), no TypeScript editing.

---

5. Safety guard

If the broadcast page and the per-channel page disagree on a register's in-page offset, codegen **aborts** (`... layout mismatch ... codegen aborted`). Writing a wrong AMax register address can corrupt the firmware, so this is intentional — fix the RegisterFile, or use an explicit override flag (`--page-base 0x... --broadcast-base 0x... --prefer-per-channel false`, decimal or `0x` hex). Overrides are rarely needed.

---

6. Apply it (restart the DAQ)

The new code lives in the rebuilt binaries; the running DAQ must be restarted to pick them up. Restart **only when no run is active** (Idle / Configured is fine — no data is lost).

```
cd /media/raid1/delila-rs
source ~/.cargo/env

scripts/stop_daq.sh
scripts/start_daq.sh config/config_amax_56_2Digitizer.toml
```

> **MongoDB:** the operator reads its connection from the config's `[operator.mongodb]` section (`--mongodb-uri` / `--mongodb-database` CLI flags are only overrides), so run history is saved **whether or not** you pass `--no-mongo`. Omitting `--no-mongo` additionally makes the script verify the persistent `delila_mongo` Docker container is up (it pings it, does not restart it) — slightly more robust, so prefer omitting it.

Check that everything is back: `http://localhost:9090/api/status` should show all components `Idle` and `online`, or open the Swagger UI at `http://localhost:9090/swagger-ui/`.

---

7. Verify on hardware

The new register-write logic takes effect on the **next Configure** after the hardware is powered on.

1. Power on the digitizer / NIM crate. (While it's off, the reader logs `CAEN error -4: DEVICE NOT FOUND` and retries — that's normal; it reconnects automatically once power is on.)
2. Configure via the Operator UI / REST (do not send raw ZMQ commands).
3. In the reader log, sanity-check the applied addresses (`BROADCAST_BASE`, per-channel addresses) match what you expect.
4. After Start, confirm the ADC spectrum / waveform looks right. If the FW misbehaves, power-cycle the digitizer and re-Configure.

---

8. Troubleshooting

Symptom	Cause / fix
`... layout mismatch ... codegen aborted`	broadcast vs per-channel offsets disagree. Check the RegisterFile; use override flags only if intentional.
`no entry in fw_params.json (skipped)`	register not exposed. Add it to `fw_params.json` and rerun (§4).
New register/tab not in the UI	`fw_params.json` entry missing, or the UI wasn't rebuilt — rerun without `--no-ui`.
`npm not found` warning	Node not on `PATH`. On gant: `source ~/.profile` (or open a new shell), then rerun.
reader `CAEN error -4: DEVICE NOT FOUND`	digitizer / crate powered off. Power on; the reader reconnects. Not a build problem.
Build error `struct AMaxChannelConfig has no field ...`	generated files out of sync with handle.rs. Rerun `scripts/update_amax_fw.sh` — it regenerates both consistently.
FW unresponsive after Configure	possible bad-address write; power-cycle the digitizer and re-Configure with correctly generated bindings.

---

9. How it works (reference)

- **Address auto-derivation** (`src/bin/amax_codegen.rs`, `derive_layout`): registers are grouped by `channel_index()` into broadcast / ch0 / chN. `PAGE_BASE` = min ch0 address (or broadcast if no per-channel pages), `PAGE_STRIDE` = `addr(ch1) - addr(ch0)`, `BROADCAST_BASE` = min broadcast address. Registers shared by both pages must keep an identical in-page offset (the safety guard, §5).

- **handle.rs is FW-agnostic**: `apply_amax_channel_config` calls the generated `channel_register_byte_addr` / `broadcast_register_byte_addr` / `channel_writes` / `merge_amax_channel_config` and never names register fields directly, so it doesn't change when registers are added or removed.
- **UI tabs are data-driven**: `channel-params.ts` iterates the generated `AMAX_PARAMS_BY_CATEGORY`, so a new category becomes a tab automatically.